

University of Stuttgart

Institute for Control Engineering of Machine
Tools and Manufacturing Units (ISW)



VHDL and embedded algorithms on an FPGA/NIOS CPU for distributed controller synchronization and motion control execution

Caren Dripke, M.Sc.

**Ahmed
Mahfouz**

About myself

- Name: Ahmed Mahfouz
- Nationality: Egyptian
- Home city: Cairo, Egypt
- University: German University in Cairo (GUC)
- Faculty: Engineering and Material Science (EMS)
- Studies: Mechatronics Engineering bachelor student

About DEVEKOS research project [1]

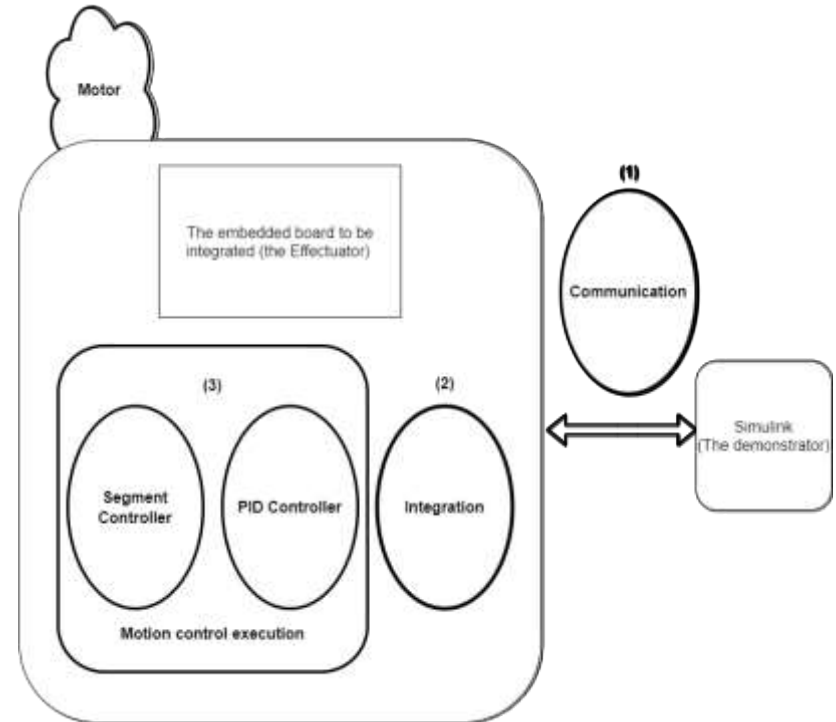
- Definition
- Started officially on: 1st of March 2017 (still on work)
- Funded by: the Federal Ministry for Economic Affairs and Energy (BMWi)



Bundesministerium
für Wirtschaft
und Energie

Presentation outline

- ✓ Introduction
- ✓ Communication implementation
- ✓ The embedded board integration into the project setup
- ✓ Motion control execution
- ✓ Conclusion and results
- ✓ Further work recommendations

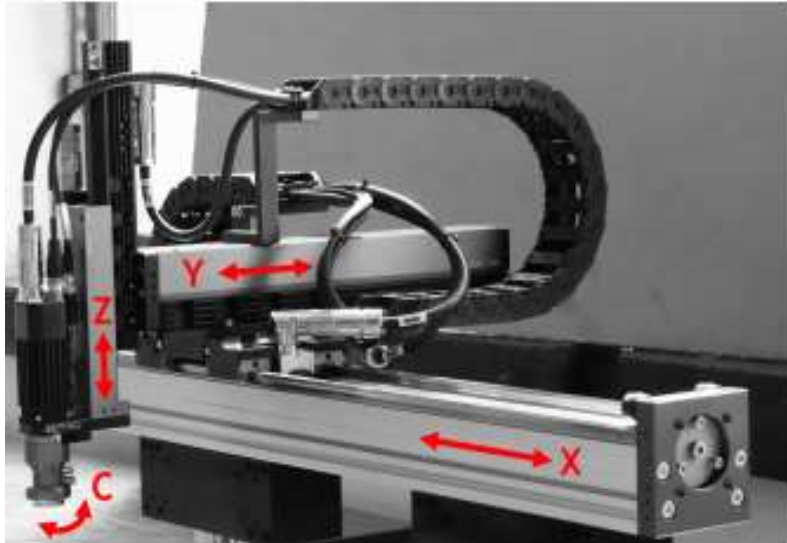


Presentation outline

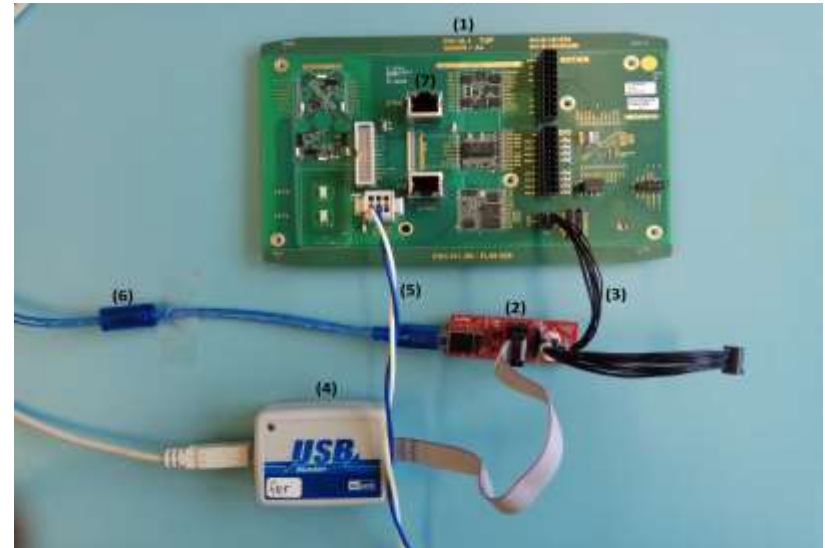
Introduction

Initial setup of my thesis project

- The demonstrator existing at ISW
- A Simulink model running on the demonstrator
- An embedded NIOS CPU (the Effectuator)
- NIOS II C project



The demonstrator existing at ISW [2]

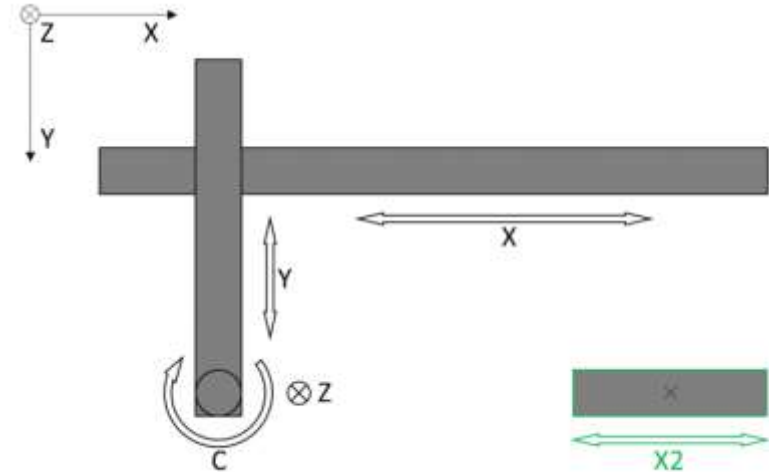


NIOS CPU board (the Effectuator) [3]

Introduction

The objective of my thesis

- Implementing a communication network between the demonstrator and the Effectuator
- Integration the Effectuator with the three transitional axes of the demonstrator
- Motion control execution for the Effectuator axis (X2-axis)



The Effectuator integrated with the demonstrator[4]

Communication implementation

The data shared between the Effectuator and the demonstrator

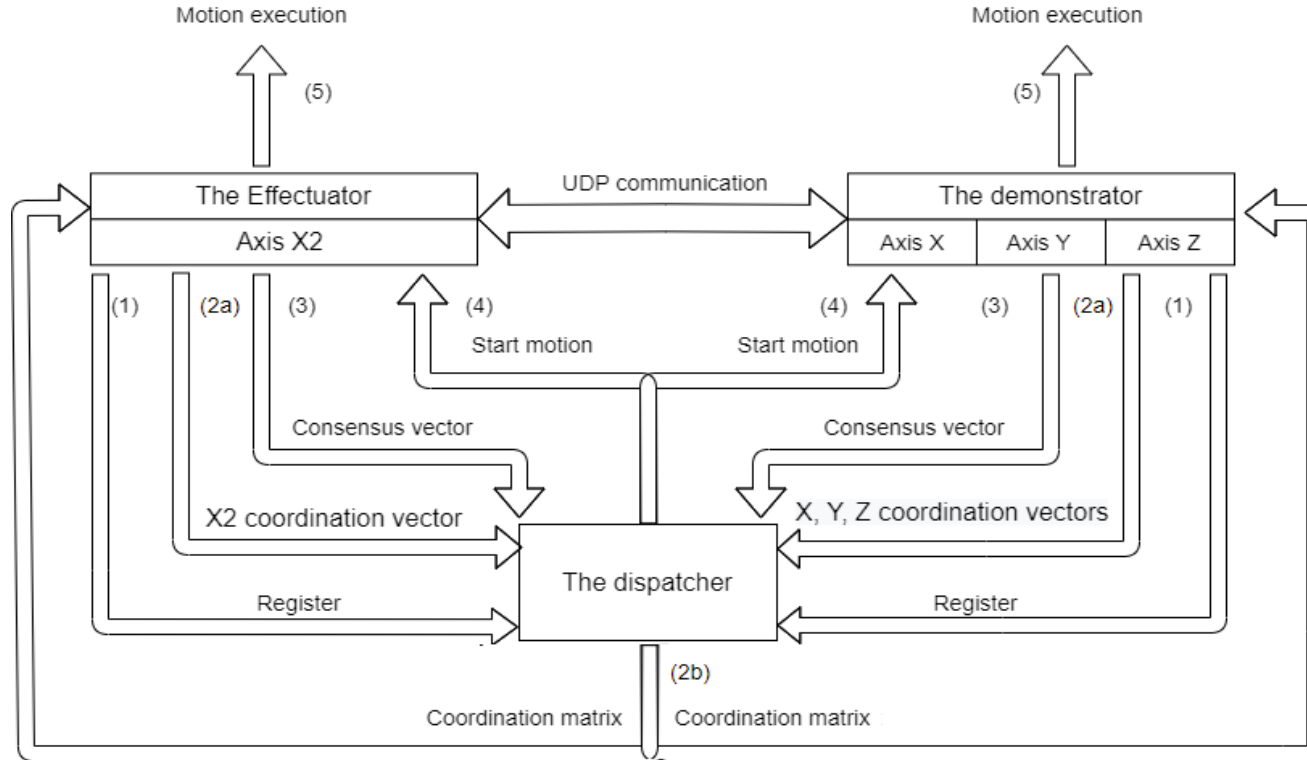
Path preparation calculations

- Coordination vector:
 - Axis 1 = $[t1.1, t1.2, t1.3, \dots, t1.n-1]$, where n is the number of points in the axis path
- Coordination matrix:
 - Axis 1 = $[t1.1, t1.2, t1.3, \dots, t1.n-1]$
 - Axis 2 = $[t2.1, t2.2, t2.3, \dots, t2.n-1]$
 - Axis 3 = $[t3.1, t3.2, t3.3, \dots, t3.n-1]$
 - Axis 4 = $[t4.1, t4.2, t4.3, \dots, t4.n-1]$
 - .
 - Axis a = $[ta.1, ta.2, ta.3, \dots, ta.n-1]$, where a is the number of the system axes

- Consensus vector:
 - $tmax.1 = \max(t1.1, t2.1, t3.1, t4.1, \dots, ta.1)$
 - $tmax.2 = \max(t1.2, t2.2, t3.2, t4.2, \dots, ta.2)$
 - $tmax.3 = \max(t1.3, t2.3, t3.3, t4.3, \dots, ta.3)$
- Consens vector= $[tmax.1, tmax.2, \dots, tmax.n-1]$
- Execution vector:
ExeVec =
[path points; speeds; times; cummulative times]

Communication implementation

The data communication sequence

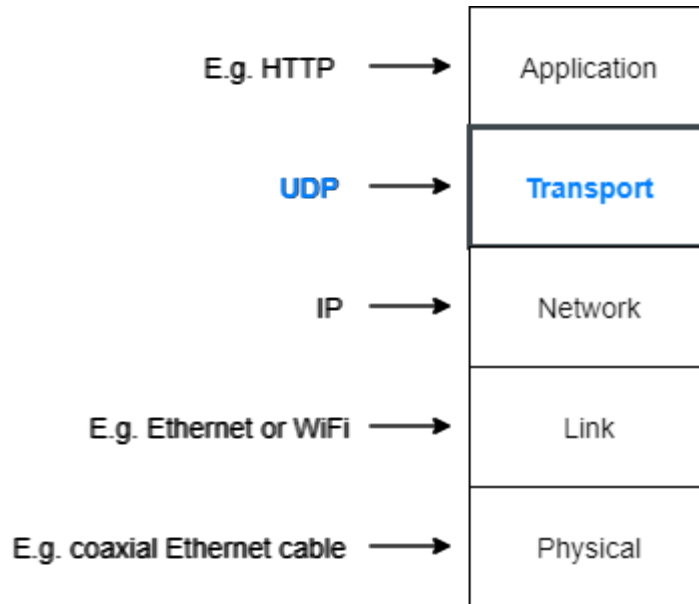


The entire process of the new system [4]

UDP communication implementation

Communication protocol and Message structure

- What is UDP/IP (User Datagram Protocol) ?



Packet structure [5]

- Message structure

Message	Data format	Data
---------	-------------	------

Schematic of message structure [4]

$$A = \begin{pmatrix} a_1 & a_2 & \dots & a_n \\ b_1 & b_2 & \dots & b_n \\ \vdots & \vdots & & \vdots \\ z_1 & z_2 & \dots & z_n \end{pmatrix}$$

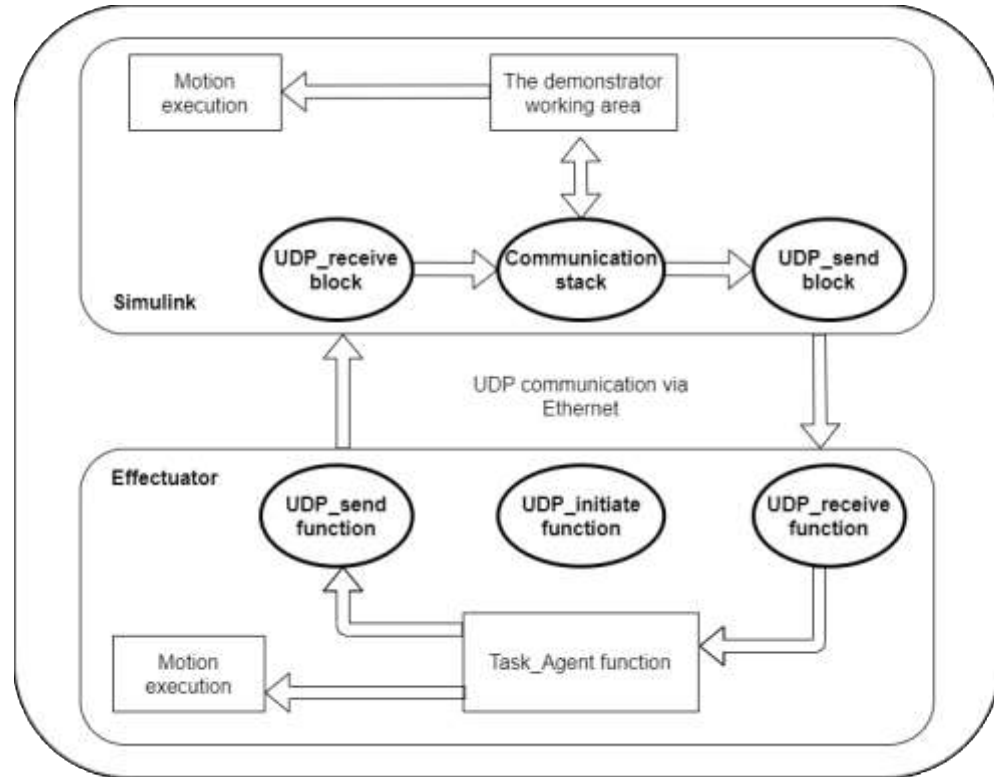
Representation of data matrix [4]

a ₁	b ₁	...	z ₁	a ₂	b ₂	...	z ₂	...	a _n	b _n	...	z _n
----------------	----------------	-----	----------------	----------------	----------------	-----	----------------	-----	----------------	----------------	-----	----------------

Schematic of the data field [4]

UDP communication implementation

- Effectuator implementation
 - UDP_initiate function
 - UDP_send function
 - UDP_receive function
- Simulink implementation
 - UDP_send block
 - UDP_receive block
 - Communication stack

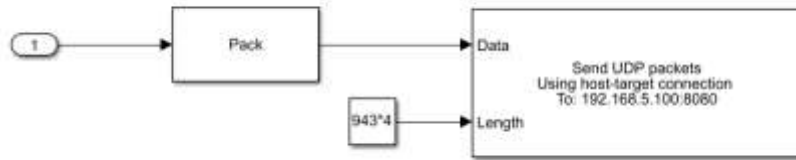


Communication Software structure

UDP communication implementation

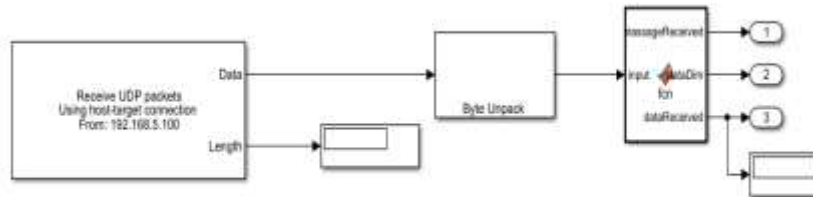
The Simulink implementation

- UDP_communication Simulink block
 - UDP_send block



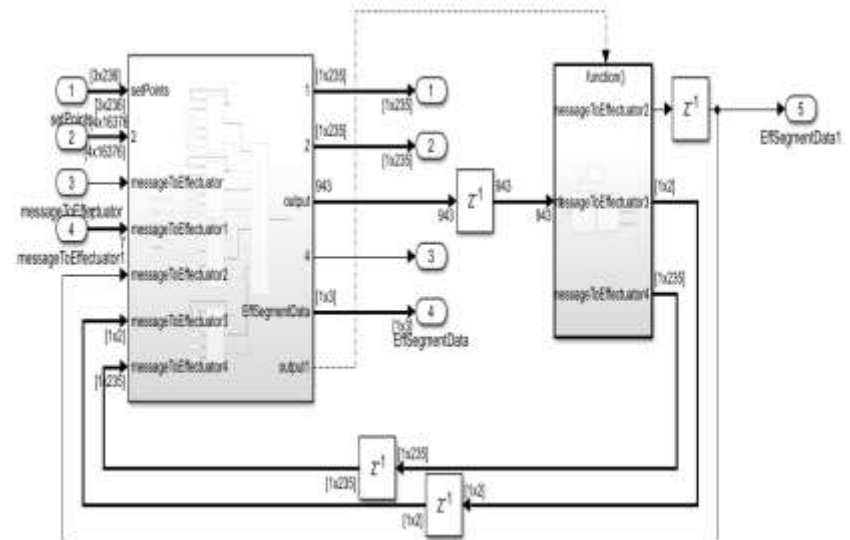
UDP_send block [4]

- UDP_receive block



UDP_receive block [4]

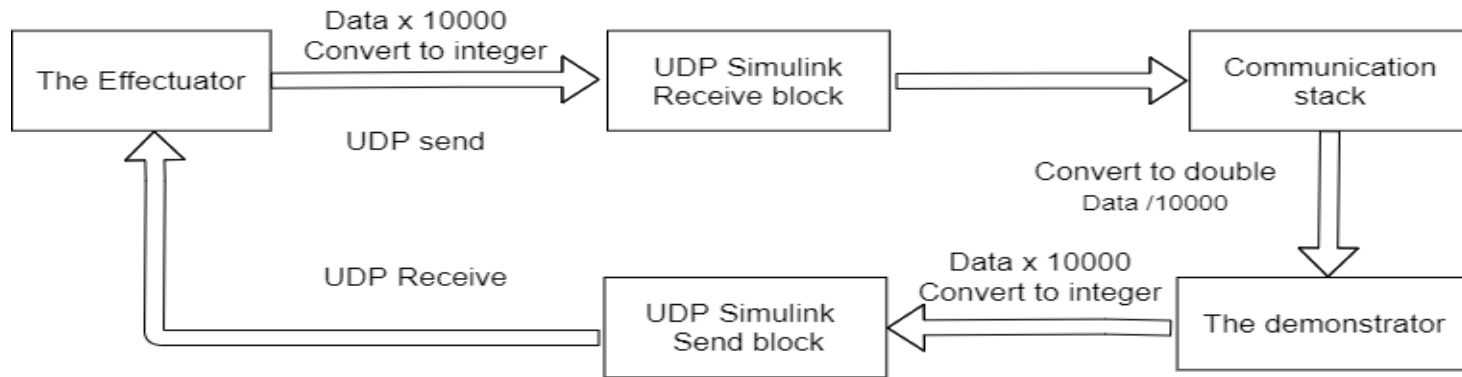
- Communication stack



Communication stack [4]

UDP communication implementation

Data type conversion

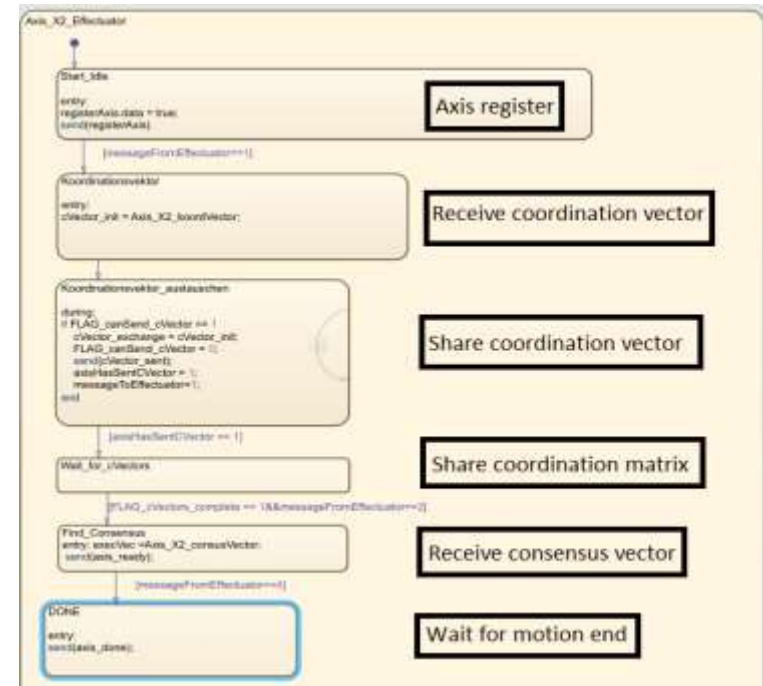


UDP communication process diagram [4]

X2-axis integration into the project setup

Inserting X2-axis into the demonstrator common working area

- The effectuator task_agent
 - Generate X2 curve
 - Generate coordination vector
 - Generate consensus vector
 - Generate execution vector
 - Execute movement
- The effectuator task_agent
 - X2-axis or the Effectuator flowchart



X2-axis flowchart [4]

X2-axis motion control implementation

PID controller embedded implementation

- Converting the transfer function of the PID into a difference equation

$$PID(z) = P + I.T_s \cdot \frac{1}{z-1} + D \cdot \frac{N}{1 + N \cdot T_s \cdot \frac{1}{z-1}} = \frac{U(z)}{E(z)} \quad (5.4)$$

Discrete PID controller transfer function [4]

Where **P** is the proportional gain, **I** is the integral gain, **D** is the derivative gain, T_s is the sampling time, **N** is the filter coefficient, **U(z)** is the transfer function output, **E(z)** is the transfer function input

X2-axis motion control implementation

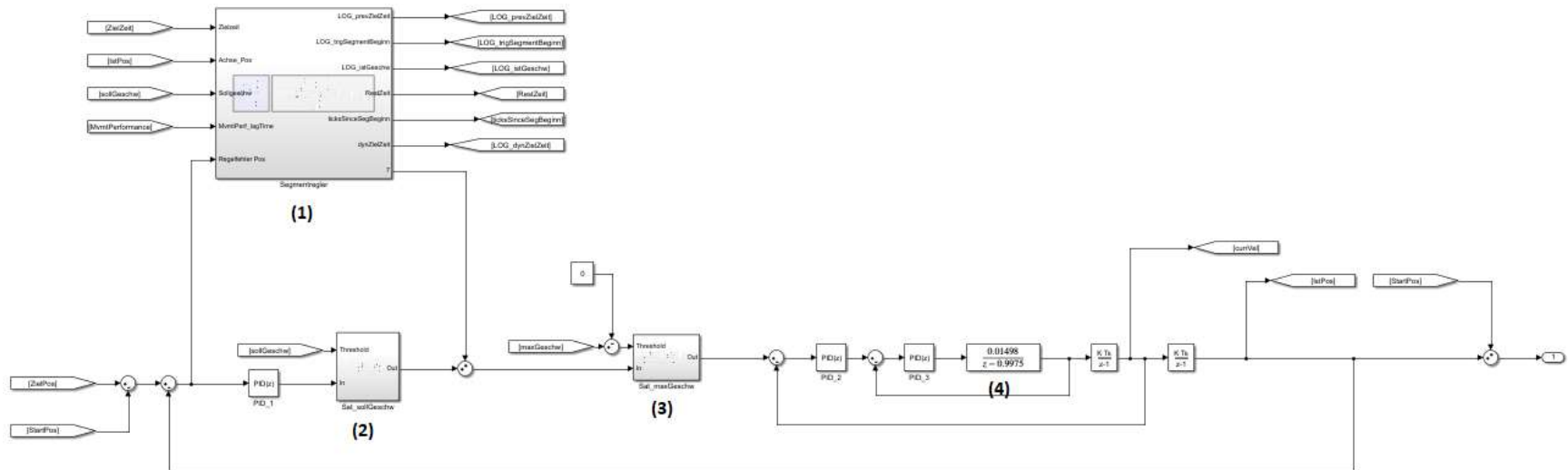
PID controller embedded implementation

$$\begin{aligned} u(k) = & (2 - N \cdot T_s)u(k-1) + (N \cdot T_s - 1)u(k-2) + (P + D \cdot N)e(k) \\ & + (P \cdot N \cdot T_s + I \cdot T_s - 2P - 2D \cdot N)e(k-1) \\ & + (P + I \cdot N \cdot T_s^2 - P \cdot N \cdot T_s - I \cdot T_s + D \cdot N)e(k-2) \end{aligned} \quad (5.5)$$

Discrete PID controller a difference equation [4]

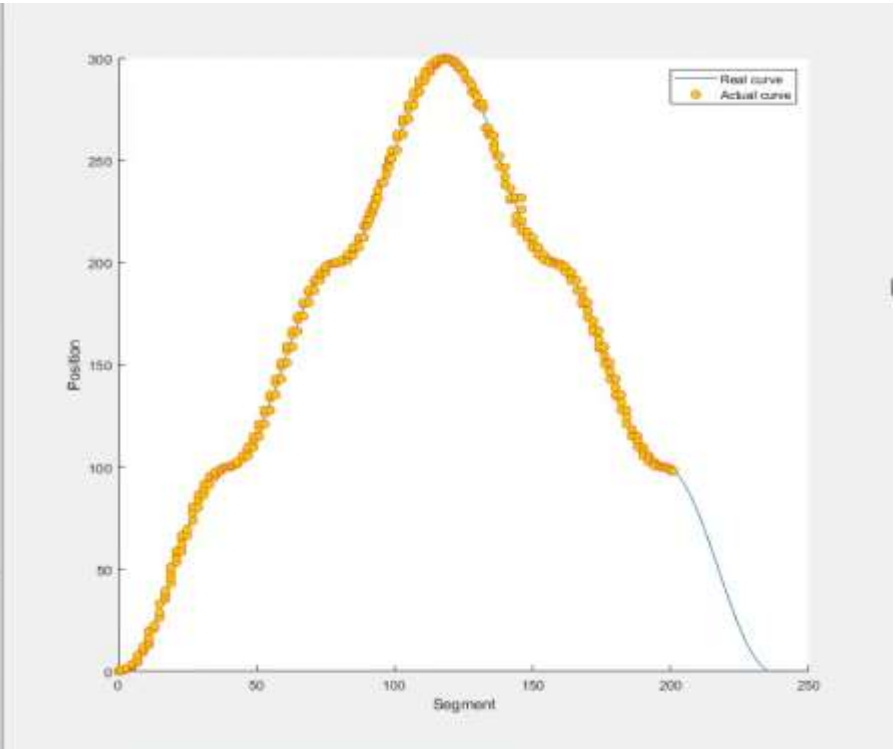
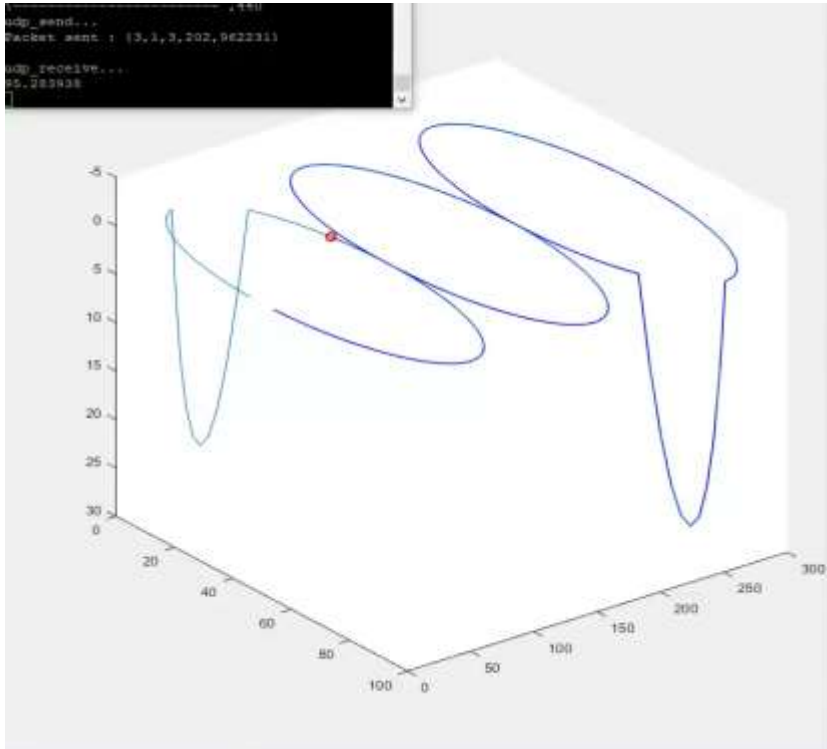
Where $\mathbf{u(k)}$ is the current PID output , $\mathbf{u(k-1)}$ is the previous PID output, $\mathbf{u(k-2)}$ is the former PID input, $\mathbf{e(k)}$ is the current PID output , $\mathbf{e(k-1)}$ is the previous PID input , $\mathbf{e(k-2)}$ is the former PID input

Axis control loop implementation



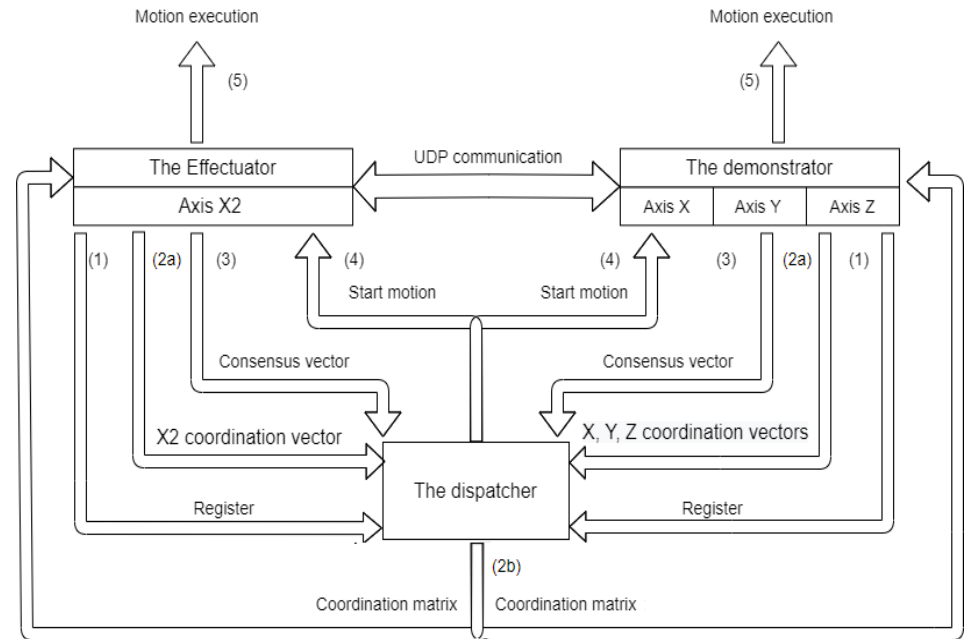
X-axis control loop [4]

Conclusion and results



Conclusion and results

- A UDP communication network
- The Effectuator integration into the demonstrator axes
- The Effectuator motion control execution



The entire process of the new system [4]

Further work recommendations

- Driving a real DC motor by the Effectuator

DB42

Brushless DC Motor

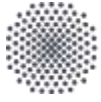
- Expanding the system with more embedded agents



DB42S03 Brushless DC Motor [6]

References

- [1] <https://www.isw.uni-stuttgart.de/forschung/engineering/devekos/>
- [2] Caren Dripke, Daniel Schoebel, and Alexander Verl. Synchronization of motion-controlled axes with coordination vector and decentralized segment controller. 9 2020.
- [3] Ahmed Mahfouz. VHDL and embedded algorithms on an FPGA/NIOS CPU for distributed controller synchronization and motion control execution. Master's thesis, Institute for Control Engineering of Machine Tools and Manufacturing Units (ISW), Stuttgart, Germany, 7 2020
- [4] Pirmin Stanglmeier. Application case study for decentralized interpolation: Integration of commercial drive components and prototypical DEVEKOS axis control components. Master's thesis, Institute for Control Engineering of Machine Tools and Manufacturing Units (ISW), Stuttgart, Germany, 1 2020
- [5] Nisha Jiju. What's the difference between ip/tcp udp? <https://www.colocationamerica.com/blog/tcp-ip-vs-udp>, December 2018.
- [6] Nanotec. DB42 Brushless BB Motor. Technical report, info@nanotec.de, Munich, Germany, 1 2020.



University of Stuttgart

Institute for Control Engineering of Machine
Tools and Manufacturing Units (ISW)



Thank you!



Ahmed Mahmoud Mohamed Aly Mahfouz

Supervisor Caren Dripke, M.Sc.

University of Stuttgart

Institute for Control Engineering of Machine Tools and Manufacturing Units (ISW)

Seidenstraße 36 • 70174 Stuttgart • Germany